

OpenTeX

Ambiente collaborativo LaTeX self-hosted su MongoDB

Relazione finale di progetto

Sommario

OpenTeX è un ambiente collaborativo self-hosted per la scrittura di documenti LaTeX, sviluppato come progetto per il corso di Basi di Dati 2 (NoSQL). Il sistema adotta MongoDB come motore di persistenza, un backend FastAPI/Motor asincrono e un frontend React/Vite, con compilazione dei documenti tramite il motore *Tectonic*. Questa relazione descrive l'installazione, l'architettura, il modello dei dati, le scelte CAP/BASE, le operazioni CRUD, la generazione di dati sintetici (ETL), gli indici e i relativi benchmark, e la pipeline di aggregazione (JOIN) realizzata sul progetto, riportando esclusivamente metriche misurate sul sistema realmente implementato.

Indice

1	Introduzione	3
1.1	Stack tecnologico	3
2	Installazione	3
2.1	Procedura	3
2.2	Servizi esposti	3
3	Architettura	4
3.1	Architettura a tre livelli	4
3.2	Modello dei dati: 5 collezioni	4
3.3	Flusso di autenticazione	4
3.4	Pipeline di compilazione LaTeX	5
4	Modello CAP e proprietà BASE	5
5	API e operazioni CRUD	6
5.1	Controllo accessi (RBAC)	6
6	ETL – Generazione di dati sintetici	6
6.1	Output di un'esecuzione reale	7
7	Indici e benchmark	7
7.1	Indici creati	7
7.2	Risultati del benchmark	7
8	Aggregation pipeline (JOIN)	8
8.1	Esempio di chiamata e risposta	8
9	Metriche di progetto	9
10	Uso dell'intelligenza artificiale	9

1 Introduzione

OpenTeX nasce per dimostrare l'uso di un database NoSQL documentale (MongoDB) in un caso d'uso realistico: un editor collaborativo di documenti LaTeX con gestione di utenti, progetti, permessi, file e log di attività. Il progetto è stato sviluppato in maniera incrementale attraverso 10 issue (vedi Sezione 11), ciascuna chiusa con una pull request dedicata e revisionata da un membro del team diverso dall'autore.

1.1 Stack tecnologico

Componente	Tecnologia	Ruolo
Frontend	React 19 + Vite 8	Single-page application
HTTP client	Axios 1.x	Iniezione automatica del Bearer JWT
Editor	@uiw/react-codemirror 4.x	Editor con syntax highlighting LaTeX
Backend	FastAPI + Motor (async)	API REST e logica applicativa
Database	MongoDB 8.0	Persistenza documentale
Compilazione	Tectonic	Compilazione LaTeX → PDF, imbarcato nel backend
Orchestrazione	Docker Compose	3 servizi: mongo, backend, frontend

Tabella 1: Stack tecnologico di OpenTeX.

2 Installazione

Il sistema è interamente containerizzato; l'unico prerequisito è Docker (≥ 24.0) e Docker Compose (≥ 2.20).

2.1 Procedura

1. Clonare il repository e configurare le variabili d'ambiente:

```
git clone <url-repo> && cd opentex
cp .env.example .env # impostare SECRET_KEY, MONGO_ROOT_USER, MONGO_ROOT_PASSWORD
```

2. Avviare tutti i servizi:

```
docker compose up -d --build
```

3. Applicare la validazione dello schema MongoDB (\$jsonSchema):

```
docker compose exec backend python -m scripts.validation.apply_schema_validation
```

4. (Opzionale) Caricare dati sintetici di esempio:

```
python seed/seed.py --users 50 --projects 100 --logs 30000 --drop
```

5. Verificare il funzionamento:

```
curl http://localhost:8000/health
```

2.2 Servizi esposti

Login di test (creato dal seed): `admin@opentex.org` / `admin1234` (account amministratore di sito, con accesso a statistiche e benchmark).

Servizio	URL	Note
Frontend	http://localhost:5173	React + Vite, hot-reload
Backend API	http://localhost:8000	FastAPI + Motor
Swagger UI	http://localhost:8000/docs	Documentazione API interattiva
MongoDB	localhost:27017	Credenziali in <code>.env</code>

Tabella 2: Servizi e porte esposte da Docker Compose.

3 Architettura

3.1 Architettura a tre livelli

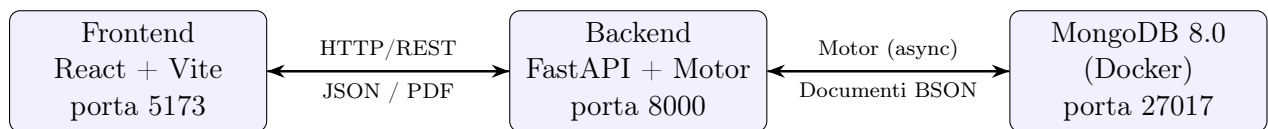


Figura 1: Architettura a tre livelli di OpenTeX. Tutti i servizi sono orchestrati via Docker Compose; il backend imbarca un binario statico di Tectonic, eliminando la necessità di un'installazione TeX separata.

3.2 Modello dei dati: 5 collezioni

Il database è organizzato in cinque collezioni. La scelta tra *embedding* e *referencing* è stata guidata da tre criteri: stabilità del dato, dimensione, e limite di 16 MB per documento imposto da MongoDB.

Decisione	Scelta	Motivazione
Metadati di progetto (tag, abstract)	Embedded	Piccoli, stabili, letti sempre insieme al progetto
File, permessi, log di attività	Referenced (ObjectId)	Crescita illimitata; i file multimediali eccedono il limite di 16 MB/documento
Profili utente	Referenced	Condivisi tra molti progetti; aggiornati indipendentemente

Tabella 3: Criteri di modellazione: embedding vs. referencing.

3.3 Flusso di autenticazione

L'autenticazione è basata su JWT firmati con algoritmo HS256. Le password sono salvate con hash `bcrypt` e non vengono mai restituite in alcuna risposta API. Il token scade dopo `JWT_EXPIRATION_MINUTES` (default 1440 minuti, 24 h).

```

POST /auth/register {email, password, first_name, last_name, department}
-> bcrypt.hash(password) salvato in users; 201 Created

POST /auth/login {email, password}
-> bcrypt.verify; JWT firmato con SECRET_KEY
-> 200 {access_token, token_type, user}

GET /projects/ (Authorization: Bearer <token>)
  
```

Collezione	Campi principali
users	email (univoca), first_name, last_name, department, hashed_password, is_admin, created_at, preferences
projects	title, abstract, tags[], owner_id → users, created_at, updated_at
files	project_id → projects, filename, file_type (tex bib image pdf), content (tex/bib) path (image/pdf), uploaded_by → users
permissions	user_id → users, project_id → projects, role (Admin Editor Viewer), granted_at, granted_by
activity_logs	user_id → users, project_id → projects, action, resource, resource_id, timestamp

Tabella 4: Schema sintetico delle 5 collezioni. La validazione `$jsonSchema` applicata in MongoDB impone i campi obbligatori, i tipi e le regole condizionali (es. `content` richiesto per `tex/bib`, `path` per `image/pdf`); non viene invece verificata l'integrità referenziale a livello di database, scelta coerente con il modello AP descritto in Sezione 4.

```
-> JWT decodificato, user_id iniettato come dipendenza
-> 200 [...]
```

3.4 Pipeline di compilazione LaTeX

La compilazione avviene in una directory temporanea isolata, con timeout e parallelismo limitato per contenere il carico sul backend:

1. Recupero di tutti i file `.tex/.bib` del progetto da MongoDB.
2. Scrittura in una directory temporanea isolata (`tempfile.mkdtemp`).
3. Acquisizione di un semaforo (massimo 2 job paralleli).
4. Esecuzione di `tectonic -keep-logs -outdir <tmp> <entry_point.tex>`, timeout 60s, `\write18` disabilitato di default.
5. Successo → lettura di `output.pdf`, risposta `application/pdf`.
6. Fallimento → risposta 422 con il log di Tectonic.
7. Timeout → risposta 504.
8. Sempre → `shutil.rmtree(tmp)` (pulizia).

L'entry point è `main.tex` se presente, altrimenti il primo file `.tex` in ordine alfabetico.

4 Modello CAP e proprietà BASE

OpenTeX gira su una singola istanza/replica MongoDB. Il sistema è progettato secondo il paradigma **AP** del teorema CAP:

- **Availability** – le richieste di lettura e scrittura hanno successo anche in presenza di guasti transitori.
- **Partition Tolerance** – il sistema continua a operare in presenza di partizioni di rete.

- **Consistency** (rilassata) – MongoDB garantisce solo atomicità a livello di singolo documento; non esistono transazioni multi-documento o cross-collection. Sono quindi possibili letture stale in casi limite (es. un documento `projects` aggiornato mentre viene letto il relativo `permissions`).

Proprietà BASE	Implementazione in OpenTeX
Basically Available	FastAPI risponde immediatamente; nessun lock globale
Soft state	Gli aggregati derivati (es. conteggio collaboratori) sono calcolati on-demand, non cacheati
Eventually consistent	I log di attività e le modifiche ai permessi si propagano alla lettura successiva; nessuna sincronizzazione write-ahead

Tabella 5: Semantica BASE adottata da OpenTeX.

5 API e operazioni CRUD

Il backend espone 7 router FastAPI con operazioni CRUD complete sulle risorse principali, documentate automaticamente via Swagger (`/docs`).

Router	Metodo	Endpoint	Descrizione
auth	POST	<code>/auth/register</code>	Registrazione utente
	POST	<code>/auth/login</code>	Login, rilascio JWT
	GET	<code>/auth/departments</code>	Elenco dipartimenti
projects	POST	<code>/projects/</code>	Creazione progetto
	GET	<code>/projects/</code>	Elenco progetti (filtro <code>owner_id</code>)
	GET	<code>/projects/{id}</code>	Dettaglio progetto
	PUT	<code>/projects/{id}</code>	Aggiornamento progetto
	DELETE	<code>/projects/{id}</code>	Cancellazione progetto
	GET	<code>/projects/{id}/files</code>	File collegati al progetto
files	PUT	<code>/files/{id}</code>	Salvataggio contenuto file
	DELETE	<code>/files/{id}</code>	Cancellazione file
permissions	POST	<code>/projects/{id}/permissions</code>	Assegna/aggiorna ruolo
	DELETE	<code>/projects/{id}/permissions/{uid}</code>	Revoca accesso
	GET	<code>/projects/{id}/permissions</code>	Elenco collaboratori
compile	POST	<code>/projects/{id}/compile</code>	Compilazione LaTeX → PDF
stats	GET	<code>/stats/projects</code>	Statistiche aggregate (JOIN)
	GET	<code>/stats/benchmarks</code>	Risultati benchmark indici
users	GET	<code>/users/</code>	Elenco utenti
	GET	<code>/users/{id}</code>	Dettaglio utente

Tabella 6: Endpoint REST implementati (15 endpoint su 7 router).

5.1 Controllo accessi (RBAC)

6 ETL – Generazione di dati sintetici

Per testare query, indici e benchmark in condizioni realistiche è stato sviluppato un generatore di dati sintetici basato su `Faker` (`seed/seed.py`), parametrico nel numero di utenti, progetti e log generati, e con integrità referenziale garantita (ogni `owner_id` punta a un utente esistente, ogni `permission` a una coppia utente-progetto valida).

```
pip install -r seed/requirements.txt
python seed/seed.py --users 50 --projects 100 --logs 30000 --drop
```

Ruolo	Read	Edit	Delete	Gestione permessi
Admin	✓	✓	✓	✓
Editor	✓	✓	—	—
Viewer	✓	—	—	—

Tabella 7: Ruoli di progetto (collezione `permissions`). Il proprietario del progetto è sempre Admin implicito ed è l'unico che può cancellarlo. Il flag `is_admin` a livello account è indipendente da questi ruoli.

6.1 Output di un'esecuzione reale

Collezione	Documenti generati
<code>users</code>	51 (incluso l'account admin)
<code>projects</code>	100
<code>files</code>	337
<code>permissions</code>	251
<code>activity_logs</code>	30 000
Tempo totale	0.3 s

Tabella 8: Risultato di un'esecuzione dello script di seed (`seed/seed_output.txt`).

7 Indici e benchmark

7.1 Indici creati

Indice	Collezione	Campi	Tipo
<code>projects_text_search</code>	<code>projects</code>	<code>title</code> , <code>abstract</code>	Full-text
<code>projects_owner_date</code>	<code>projects</code>	<code>owner_id</code> <code>created_at</code>	+ Composto
<code>permissions_project_id</code>	<code>permissions</code>	<code>project_id</code>	Singolo campo
<code>activity_logs_project_id</code>	<code>activity_logs</code>	<code>project_id</code>	Singolo campo
<code>files_project_id</code>	<code>files</code>	<code>project_id</code>	Singolo campo

Tabella 9: Indici creati da `scripts/indexes/create_indexes.py`, verificati con `explain()`.

7.2 Risultati del benchmark

Le query sono state misurate con e senza l'indice corrispondente, su 10 esecuzioni per query (media riportata).

Query	Con indice	Senza indice	Speedup
Text search (<code>title</code> + <code>abstract</code>)	1.753 ms	2.003 ms	1.1×
Compound: <code>owner_id</code> + <code>created_at</code>	1.492 ms	1.655 ms	1.1×
Permissions by <code>project_id</code>	1.561 ms	1.667 ms	1.1×
Activity logs by <code>project_id</code>	1.750 ms	19.024 ms	10.9×
Files by <code>project_id</code>	1.609 ms	1.979 ms	1.2×

Tabella 10: Tempi medi di esecuzione su 10 run (`scripts/benchmark/run_benchmark.py`).

Interpretazione. Lo speedup più marcato ($10.9\times$) si osserva sulla query su `activity_logs`, l'unica collezione realmente di grandi dimensioni (30 000 documenti): senza indice MongoDB esegue una scansione completa (`COLLSCAN`) su tutti i 30k documenti, con indice risolve la stessa query in $O(\log n)$ (`IXSCAN`). Le altre query mostrano guadagni modesti ($1.1\text{--}1.2\times$) perché le collezioni coinvolte sono piccole (≤ 356 documenti): a questa scala `COLLSCAN` e `IXSCAN` hanno tempi di esecuzione comparabili, e l'overhead della rete Docker domina la misura. Lo speedup di questi indici crescerebbe proporzionalmente al volume di dati.

8 Aggregation pipeline (JOIN)

L'endpoint `GET /stats/projects` realizza una pipeline di aggregazione multi-stadio che unisce quattro collezioni, rappresentando la principale dimostrazione di JOIN cross-collection in MongoDB.

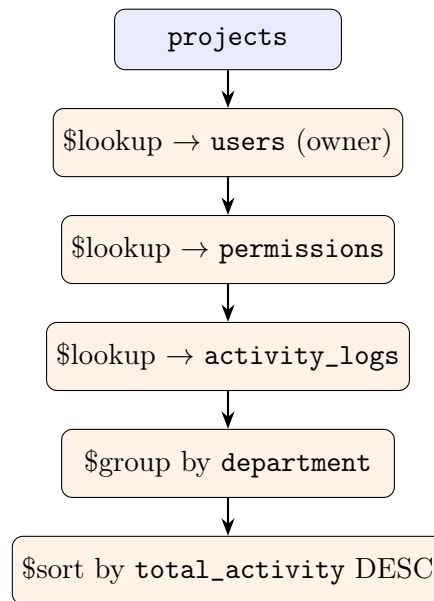


Figura 2: Pipeline di aggregazione di `/stats/projects`. Filtri opzionali (`department`, `date_from`, `date_to`) sono applicati in uno stadio `$match` prima dei lookup.

8.1 Esempio di chiamata e risposta

```
GET /stats/projects?department=Engineering
```

```
[
  {
    "department": "Engineering",
    "project_count": 12,
    "total_collaborators": 34,
    "avg_collaborators": 2.83,
    "total_activity": 1520
  }
]
```


Area	Righe di codice (LOC)
Backend Python (backend/)	1 245
Frontend JS/JSX (frontend/src/)	1 466
Script seed + scripts (indici/benchmark/validazione)	1 024
Totale	3 735

Tabella 11: Conteggio righe di codice (`wc -l` sui file `.py/.js/.jsx`, esclusi `node_modules`).

Metrica	Valore
Collezioni MongoDB	5
Indici creati	5
Router API	7
Endpoint REST	15
Ruoli RBAC	3 (Admin / Editor / Viewer)
Pagine frontend	6
Componenti UI condivisi	4

Tabella 12: Metriche riassuntive di progetto.

9 Metriche di progetto

10 Uso dell'intelligenza artificiale

In conformità con i requisiti del corso, l'uso di strumenti di intelligenza artificiale generativa è dichiarato in modo trasparente nei form di valutazione associati a ciascuna issue (cartella 10/Form/):

- **Issue #2** (generazione dati sintetici) – assistente Claude usato come supporto alla generazione dello script di seed; output verificato ed eseguito prima del merge.
- **Issue #10** (documentazione finale) – Claude Code usato per la generazione della documentazione, il fix del Dockerfile (build-arg `TARGETARCH`), le traduzioni e il polish del frontend; output verificato prima del merge.

In tutti i casi il codice e i contenuti generati sono stati revisionati da un membro del team prima dell'integrazione, secondo il workflow standard: 1 issue → 1 branch → 1 PR → 1 review → merge.

11 Conclusioni

Il progetto OpenTeX è stato sviluppato attraverso 10 issue, ciascuna chiusa con una pull request dedicata:

1. Schema e validation delle 5 collezioni MongoDB
2. Generatore di dati sintetici (seed)
3. Setup repository, Docker Compose e configurazione base
4. API CRUD per la gestione dei progetti
5. Sistema di permessi e gestione collaboratori

6. UI Dashboard "I miei Progetti"
7. Aggregation pipeline per statistiche (JOIN tra collezioni)
8. Creazione e ottimizzazione indici
9. Benchmark query con e senza indici
10. Documentazione finale e proof of concept

Tutte le issue risultano chiuse e le relative pull request mergiate su `main` (l'ultima, PR #28, relativa alla documentazione finale). Il sistema è interamente riproducibile da zero seguendo le istruzioni del README, non contiene credenziali reali nel repository, ed è stato verificato end-to-end (registrazione, login, creazione progetto, upload/editing file, compilazione LaTeX, gestione permessi, consultazione statistiche).